

Karen E2E Tests - Playwright

End-to-end tests for Karen AI using Playwright to verify Gemini live responses and prevent logout bugs.

Setup

1. Install Dependencies

```
cd tests/e2e
npm install
```

2. Install Playwright Browsers

```
npm run install
# or
npx playwright install chromium
```

3. Configure Environment

Create `.env` file in `tests/e2e/`:

```
KAREN_BASE_URL=http://localhost:8010
KAREN_API_URL=http://localhost:8000
```

4. Ensure Services Running

```
# From project root
docker compose up -d
```

5. Verify Gemini API Key

Ensure `GEMINI_API_KEY` is set in main `.env` file:

```
# In /mnt/Development/KIRO/AI-Karen/.env
GEMINI_API_KEY=your_actual_gemini_api_key_here
```

Running Tests

Run All Tests

```
npm test
```

Run Gemini Tests Only

```
npm run test:gemini
```

Run with UI (Interactive Mode)

```
npm run test:ui
```

Run in Headed Mode (See Browser)

```
npm run test:headed
```

Debug Mode

```
npm run test:debug
```

View Test Report

```
npm run report
```

PROF

Test Scenarios

1. Gemini Live Response Test

- **Purpose:** Verify Gemini returns live responses (not fallback)
- **Steps:**
 1. Login with admin@kari.ai / Admin@123!
 2. Select Gemini provider
 3. Send test message
 4. Verify response is live (not degraded/fallback)
 5. Check metadata shows Gemini as actual provider

2. No Logout Bug Test

- **Purpose:** Verify users stay logged in after multiple messages
- **Steps:**
 1. Login
 2. Send first message
 3. Wait for response
 4. Send second message
 5. Verify still logged in (not redirected to login page)

3. API Key Configuration Test

- **Purpose:** Verify Gemini provider is properly configured
- **Steps:**
 1. Check `/api/health/providers/all` endpoint
 2. Verify Gemini provider exists and is enabled

4. Metadata Verification Test

- **Purpose:** Verify response metadata shows `live_model` source
- **Steps:**
 1. Send message
 2. Open response details
 3. Verify `response_source: live_model`
 4. Verify `actual_provider` matches `requested_provider`

Expected Results

✓ Pass Criteria

1. Gemini Response:

- Response text is not empty
- Response length > 10 characters
- Does NOT contain: "Karen could not reach", "Emergency fallback", "degraded mode", "unavailable"

2. Metadata:

- Contains "gemini" as provider
- Does NOT contain: "builtin_vllm", "builtin_transformers", "fallback"
- `response_source: "live_model"`
- `actual_provider` matches `requested_provider`

3. No Logout:

- User remains logged in after multiple messages
- Chat interface stays visible

- URL does not redirect to /login or /signin

✗ Fail Criteria

1. Response contains degraded/fallback messages
2. Metadata shows fallback provider instead of Gemini
3. User gets logged out after sending message
4. Response is empty or too short
5. Gemini provider not configured/enabled

Troubleshooting

Test Fails: "Gemini unavailable"

Cause: Gemini API key not configured or invalid

Fix:

```
# Check API key in main .env
cat /mnt/Development/KIRO/AI-Karen/.env | grep GEMINI_API_KEY

# Restart API to load new key
docker compose restart api

# Verify provider health
curl http://localhost:8000/api/health/providers/gemini | jq
```

Test Fails: "Cannot find login form"

Cause: Frontend not running or wrong URL

Fix:

```
# Check frontend is running
curl http://localhost:3000

# Check docker services
docker compose ps

# Restart web service
docker compose restart web
```

Test Fails: "Timeout waiting for response"

Cause: Gemini API slow or rate limited

Fix:

- Increase timeout in test (already set to 30s)
- Check Gemini API status
- Verify API key has quota remaining

Test Fails: "User logged out"

Cause: Session/token expiration or authentication bug

Fix:

1. Check browser console for errors (F12)
2. Check API logs: `docker compose logs api | grep -i "auth\|token"`
3. Verify session timeout settings
4. Check WebSocket connection state

CI/CD Integration

GitHub Actions Example

```
name: E2E Tests

on: [push, pull_request]

jobs:
  e2e:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Setup Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '18'

      - name: Install dependencies
        run: |
          cd tests/e2e
          npm install
          npx playwright install chromium

      - name: Start services
        run: docker compose up -d

      - name: Wait for services
        run: |
          timeout 60 bash -c 'until curl -f http://localhost:3000; do
sleep 2; done'
          timeout 60 bash -c 'until curl -f
```

```
http://localhost:8000/health; do sleep 2; done'
```

```
- name: Run E2E tests
  env:
    KAREN_BASE_URL: http://localhost:3000
    KAREN_API_URL: http://localhost:8000
    GEMINI_API_KEY: ${ secrets.GEMINI_API_KEY }
  run: |
    cd tests/e2e
    npm test

- name: Upload test results
  if: always()
  uses: actions/upload-artifact@v3
  with:
    name: playwright-report
    path: tests/e2e/playwright-report/
```

Test Maintenance

Adding New Tests

1. Create new `.spec.ts` file in `tests/e2e/`
2. Import Playwright test utilities
3. Follow existing test patterns
4. Add test to npm scripts if needed

Updating Selectors

If UI changes break tests, update selectors in test files:

```
// Old selector
await page.locator('button:has-text("Send")').click();

// New selector (more robust)
await page.locator('[data-testid="send-button"]').click();
```

Best Practices

1. Use data-testid attributes for stable selectors
2. Wait for elements before interacting
3. Use explicit waits, not arbitrary timeouts
4. Take screenshots on failure
5. Keep tests independent and idempotent

Support

For issues or questions:

- Check troubleshooting section above
 - Review Playwright documentation: <https://playwright.dev>
 - Check Karen documentation in [/docs](#)
 - Review API logs: [docker compose logs api](#)
-

Last Updated: 2026-04-27

Test Framework: Playwright v1.40+

Node Version: 18+

Login Credentials: admin@kari.ai / Admin@123!